

How byDebut gets built in one sprint

The implementation contract for the frozen MVP: stack, system flow, the full Postgres schema with row-level security, the one Edge Function that calls Gemini, realtime wiring, and the acceptance checks that say "done".

- 4 TABLES
- 1 EDGE FUNCTION
- 5 ROUTES
- REALTIME

01 · STACK

Everything Google-native, nothing extra

Frontend

React 18 + TypeScript, built with Vite. Tailwind for layout, brand tokens inline. `@supabase/supabase-js` for data + auth + realtime. Hosted as a static bundle on Cloud Run / any CDN.

Backend · Supabase

Postgres (data + RLS), Supabase Auth (magic-link), Realtime (tree, log, metrics), and one Deno Edge Function `generate-version` that owns the Gemini call.

AI · Gemini

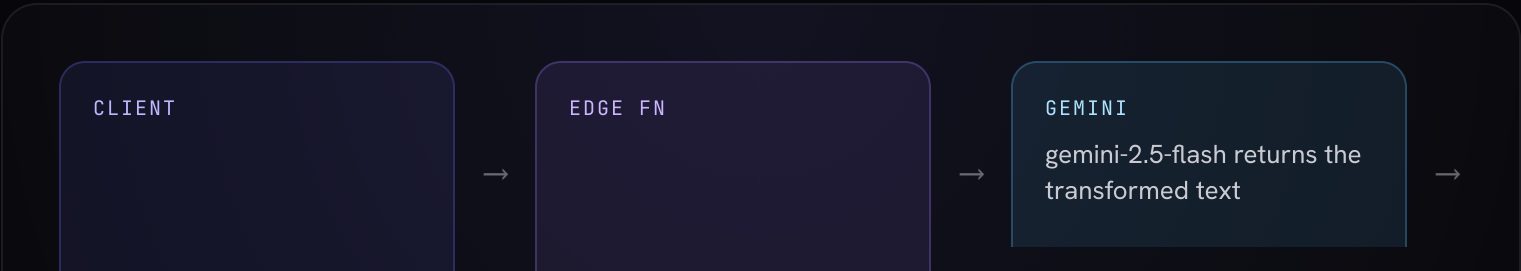
`gemini-2.5-flash` via the official SDK, called `server-side only` from the Edge Function. Streamed response; key lives in a Supabase secret, never shipped to the browser.

Ops

Schema + seed in versioned SQL migrations. Secrets via `supabase secrets`. No queue, no cache, no extra services — the demo doesn't need them.

02 · SYSTEM FLOW

One write fans out to three views



Creator Studio — choose transformation, press Generate

generate-version: build prompt, call Gemini, stream tokens

POSTGRES

INSERT versions + INSERT ai_events (one transaction)

REALTIME ⚡ Postgres change events push to three subscribed views simultaneously →

Version Tree

AI Activity Log

Metrics

03 · DATABASE SCHEMA

Four tables. One self-reference is the whole tree.

Postgres DDL, copy-paste-able as the first migration. `versions.parent_id` pointing back at `versions.id` is the Version Tree — no separate edge table.

profiles

```
id uuid PK→auth
email text
display_name text
created_at timestampz
```

books

```
id uuid PK
title · author text
body text
is_public_domain bool
created_at timestampz
```

versions ↻ self

```
id uuid PK
book_id →books
parent_id →versions
creator_id →profiles
kind · title · body
status · created_at
```

ai_events

```
id uuid PK
version_id →versions
agent · action text
model · status
latency_ms int
created_at timestampz
```

supabase/migrations/0001_init.sql

```
-- profiles: mirrors auth.users, created by trigger on signupcreate table profiles (  
  id          uuid primary key references auth.users on delete cascade,  
  email       text,  
  display_name text,
```

```

    created_at    timestampz not null default now()
);

-- books: seeded public-domain works (MVP = Frankenstein only)create table books (
    id            uuid primary key default gen_random_uuid(),
    title         text not null,
    author        text not null,
    body          text not null,
    is_public_domain boolean not null default true,
    created_at    timestampz not null default now()
);

-- versions: a branch. parent_id self-FK IS the Version Tree.create type version_kind as enum (
    'alternate_ending', 'children', 'spanish', 'educational'
);
create type version_status as enum ('generating', 'ready', 'failed');

create table versions (
    id            uuid primary key default gen_random_uuid(),
    book_id       uuid not null references books on delete cascade,
    parent_id     uuid references versions on delete cascade, -- null = root
    creator_id    uuid not null references profiles,
    kind          version_kind not null,
    title         text not null,
    body          text,
    status        version_status not null default 'generating',
    created_at    timestampz not null default now()
);

create index on versions (book_id);
create index on versions (parent_id);

-- ai_events: one row per Gemini call. Powers the activity log + metrics.create table ai_events (
    id            uuid primary key default gen_random_uuid(),
    version_id    uuid references versions on delete cascade,
    agent         text not null default 'Creator Agent',
    action        text not null,
    model         text not null default 'gemini-2.5-flash',
    status        text not null default 'success',
    latency_ms    integer,
    created_at    timestampz not null default now()
);
create index on ai_events (created_at desc);

```

Read-open for the demo, write-gated to the Edge Function

Any signed-in user can read everything (the tree, the log, the metrics are public to the demo). Writes to **versions** and **ai_events** happen only through the Edge Function's service role, so the client can't fabricate data.

0002_rls.sql

```
alter table books, versions, ai_events, profiles enable row level security;

-- everyone signed in can READ (demo is fully visible)
create policy "read books" on books for select to authenticated using (true);
create policy "read versions" on versions for select to authenticated using (true);
create policy "read events" on ai_events for select to authenticated using (true);

-- profiles: a user sees & edits only their own row
create policy "own profile" on profiles for all to authenticated using (id = auth.uid());

-- NO client insert/update policies on versions or ai_events.-- the Edge Function uses the service
```

05 · THE CORE · generate-version

The one function that has to be real

A Deno Edge Function. Receives a book + transformation, calls Gemini server-side, streams text to the client, and persists the result. This is the feature the whole demo rests on.

REQUEST / RESPONSE

```
POST /functions/v1/generate-versionAuthorization
{"book_id": "uuid",
 "kind": "children"} → 200, text/event-stream
```

STEPS

- 1 Verify JWT, resolve **creator_id** from the session.
- 2 Load **books.body**, take a bounded excerpt (token budget).
- 3 Insert a **versions** row, **status='generating'** → get id.
- 4 Call Gemini with the prompt template; stream tokens to client.
- 5 On finish: update the row **body + status='ready'**.
- 6 Insert **ai_events** with **latency_ms**. Realtime fans out.

prompt template · per transformation

```
const PROMPTS = {
  children: "Rewrite this excerpt as a gentle children's edition (ages 8+). Simple words, keep",
  spanish: "Translate this excerpt into natural literary Spanish, adapting idioms for tone.",
  educational: "Produce a grade 9-12 study edition: the passage plus 3 discussion questions and ke",
  alternate_ending: "Continue this excerpt with a compelling ALTERNATE ending that stays consisten",
};

const prompt = `${PROMPTS[kind]}\n\n--- SOURCE ---\n${excerpt}`;
const stream = await genAI
  .getGenerativeModel({ model: "gemini-2.5-flash" })
  .generateContentStream(prompt);
```

GEMINI_API_KEY is set with `supabase secrets set` and read via `Deno.env` — it is never bundled into the frontend.

06 · FRONTEND MAP

Five routes, each bound to its rows

<code>/login</code>	Magic-link sign-in; redirect to <code>/library</code> on session.	<code>supabase.auth</code>
---------------------	---	----------------------------

<code>/library</code>	Frankenstein card → reader view of <code>books.body</code> .	<code>select books</code>
-----------------------	--	---------------------------

<code>/studio</code>	Pick transformation → call <code>generate-version</code> , render the stream.	<code>invoke edge fn</code>
----------------------	---	-----------------------------

<code>/tree</code>	Constellation from <code>versions</code> where <code>book_id = Frankenstein</code> .	<code>select + realtime</code>
--------------------	--	--------------------------------

<code>/ops</code>	AI activity log (<code>ai_events</code>) + four <code>count()</code> metrics, side by side.	<code>select + realtime</code>
-------------------	---	--------------------------------

Realtime channels: the `/tree` and `/ops` routes subscribe to `postgres_changes` on `versions` and `ai_events`. The moment the Edge Function inserts, every open view re-renders — no manual refetch, no polling.

07 · METRICS · HONEST COUNTS

```
-- each card = one count, nothing fakedselect
```

SEED + SECRETS

- Seed Frankenstein — one `insert` into `books` with Project Gutenberg text (a few chapters) in `0003_seed.sql`.

Exposed as a Postgres `view` (`mvp_metrics`) so the client makes one read. Small, true numbers — that's the proof.

- Root node — the book itself is the tree root; versions hang off it via `parent_id = null` on first-level branches.
- `supabase secrets set GEMINI_API_KEY=...` — the only secret the app needs.
- `trigger` on `auth.users` → auto-creates the matching `profiles` row on signup.

08 · ACCEPTANCE

The MVP is done when...

✓ A real session persists across reload and gates every route.

✓ Frankenstein opens, reading from the `books` table.

✓ Choosing a transformation triggers a real Gemini call whose text is persisted.

✓ The new branch appears in the tree and opens to its generated text.

✓ Every generation writes one `ai_events` row, visible in the log within a second.

✓ Each metric equals a row count you can verify directly in the DB.

byDebut · technical plan & schema — frozen MVP, 7 features. Companion to the MVP scope plan.